

Projeto e Análise de Algoritmos

Buscas em grafos: Busca em Profundidade

Atílio G. Luiz

Segundo Semestre de 2025

Busca em profundidade

Busca em largura $\times$ busca em profundidade

Na busca em largura:

- ▶ Dado o conjunto não vazio Q de vértices alcançados pela origem s e ainda não finalizados, selecionamos o vértice que está há **mais tempo** em Q para continuar a busca.
- ▶ Logo, Q é uma **fila**.

Na busca em profundidade:

- ▶ Dado o conjunto não vazio Q de vértices alcançados pela origem s e ainda não finalizados, selecionamos o vértice que está há **menos tempo** em Q para continuar a busca.
- ▶ Logo, Q é uma **pilha**.

Busca em profundidade

Ideia do algoritmo

- ▶ começamos com o vértice de origem s
- ▶ para cada vizinho não visitado v do vértice atual
 1. adicionamos uma aresta (u, v) à árvore de busca
 2. visitamos **recursivamente** a partir de v

Floresta de busca

Visitando todos os vértices

- ▶ a árvore de busca contém só vértices alcançáveis de s
- ▶ algumas vezes queremos visitar todos os vértices
- ▶ repetimos o processo com os vértices não visitados
- ▶ obteremos uma **floresta de busca**

Representando uma floresta

- ▶ também utilizamos um vetor de pais π
- ▶ um vértice v com $\pi[v] = \text{NIL}$ é raiz de uma árvore de busca
- ▶ Dada uma floresta em profundidade G_π de um grafo G , temos que $V(G_\pi) = V(G)$ e as arestas da floresta são

$$E(G_\pi) = \{(\pi[v], v) : v \in V(G) \text{ e } \pi[v] \neq \text{NIL}\}$$

Cores dos vértices

De novo, vamos pintar o grafo durante a busca

1. $cor[v] = \text{branco}$ se não descobrimos v ainda
2. $cor[v] = \text{cinza}$ se já descobrimos, mas não finalizamos v
3. $cor[v] = \text{preto}$ se já descobrimos e já finalizamos v

Observações

- ▶ os vértices cinza têm suas chamadas recursivas ativas
- ▶ a pilha de chamadas induz um caminho na floresta

Tempo de descoberta e finalização

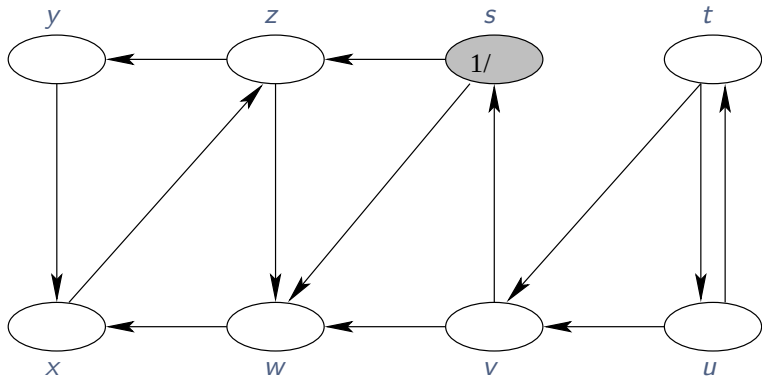
Vamos associar rótulos aos vértices

- ▶ $d[v]$ é instante de descoberta de v
- ▶ $f[v]$ é instante de finalização de v

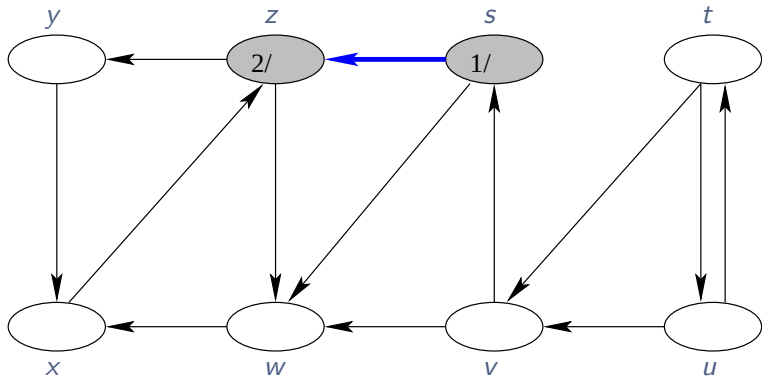
Observações

- ▶ os rótulos são inteiros distintos entre 1 e $2|V|$
- ▶ refletem os instantes em que v muda de cor

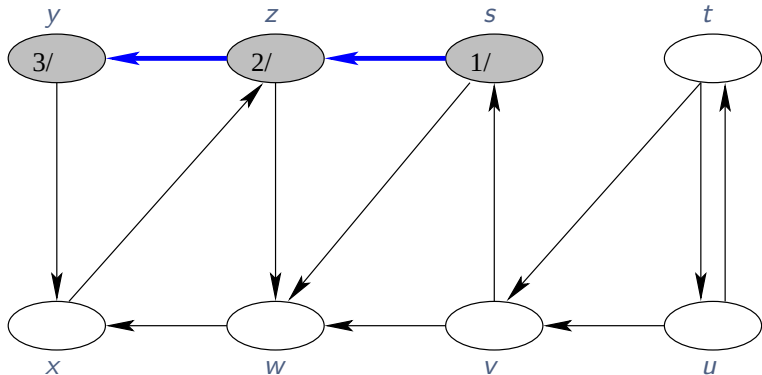
Exemplo de busca em profundidade



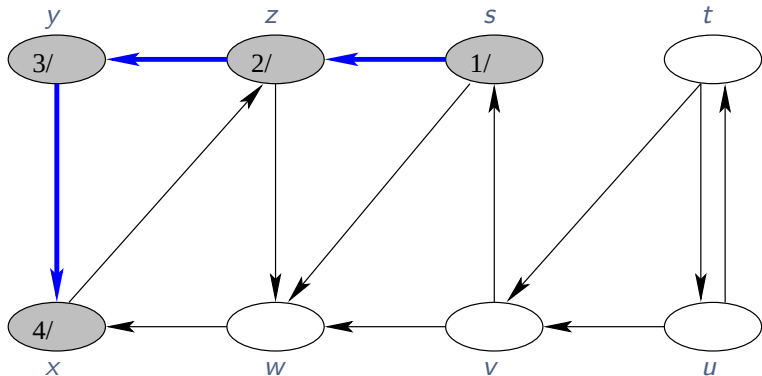
Exemplo de busca em profundidade



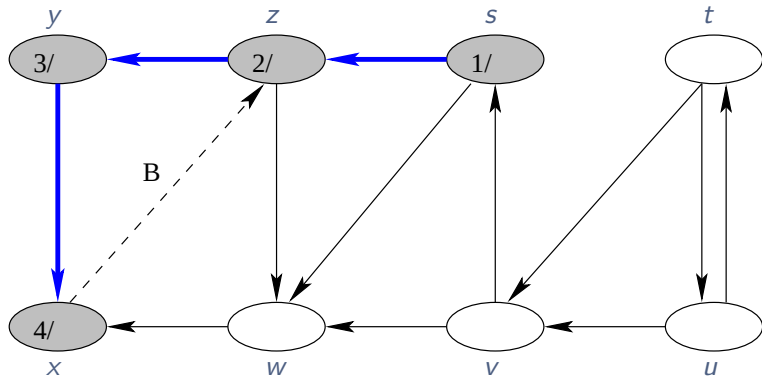
Exemplo de busca em profundidade



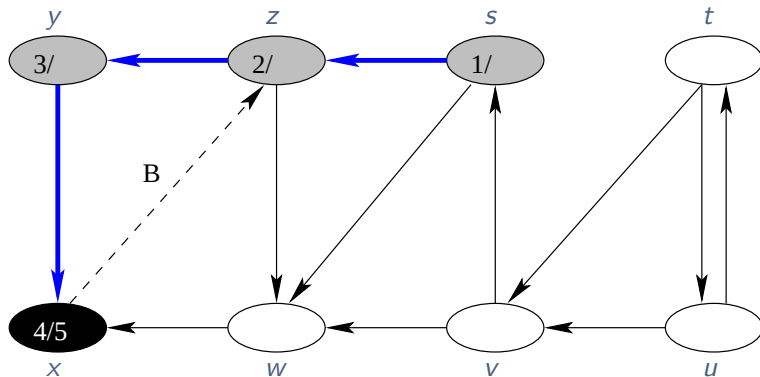
Exemplo de busca em profundidade



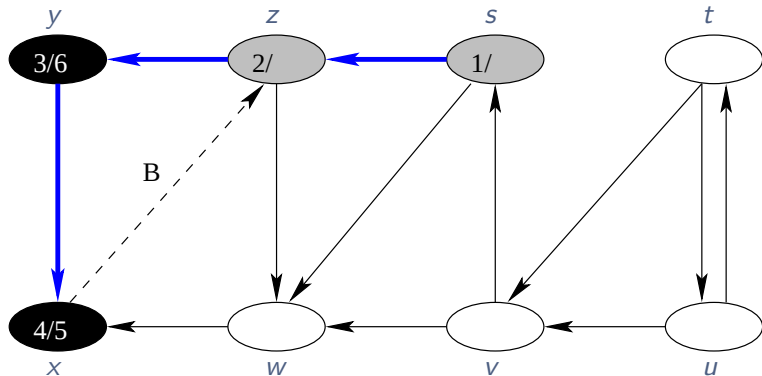
Exemplo de busca em profundidade



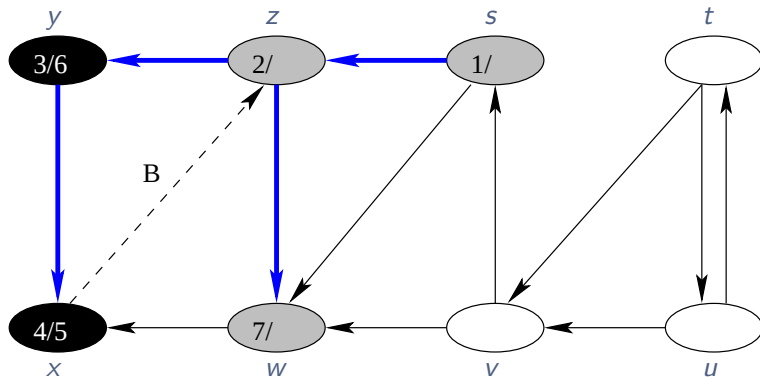
Exemplo de busca em profundidade



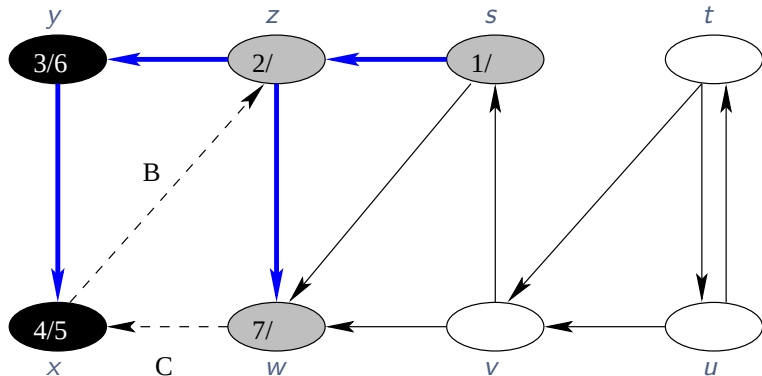
Exemplo de busca em profundidade



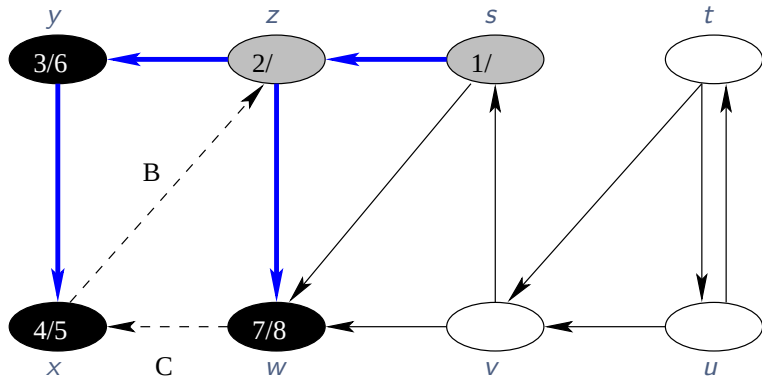
Exemplo de busca em profundidade



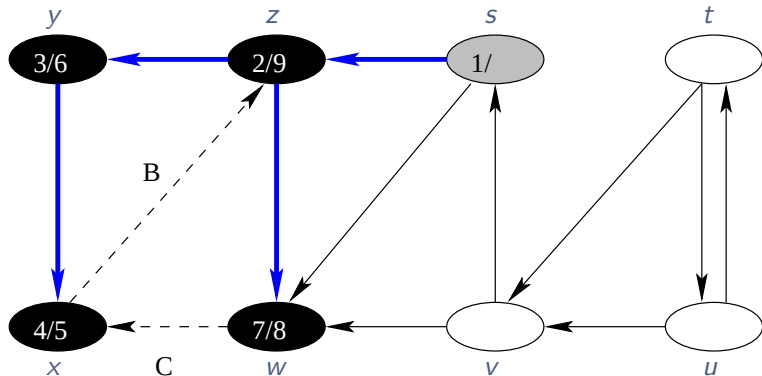
Exemplo de busca em profundidade



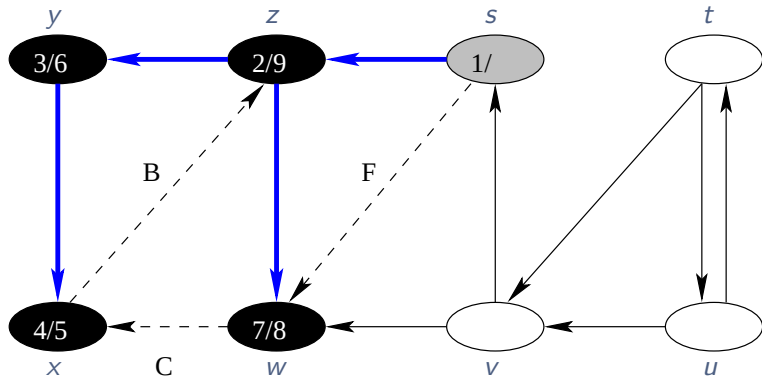
Exemplo de busca em profundidade



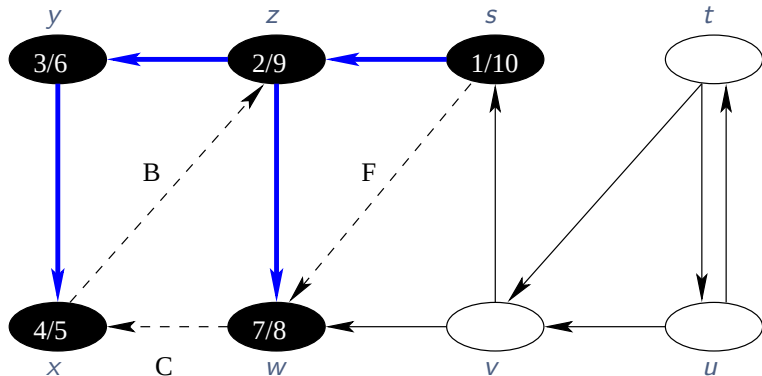
Exemplo de busca em profundidade



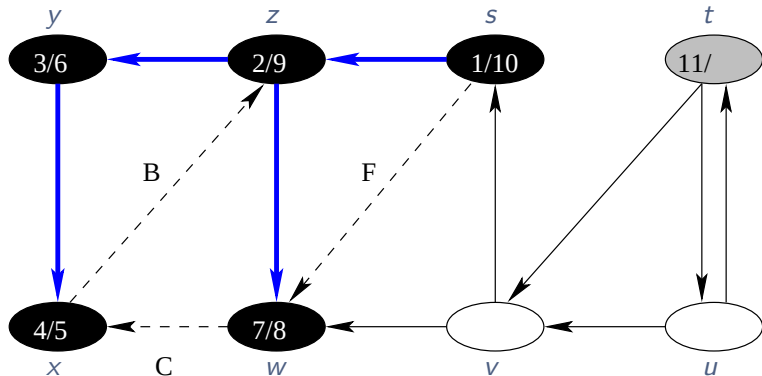
Exemplo de busca em profundidade



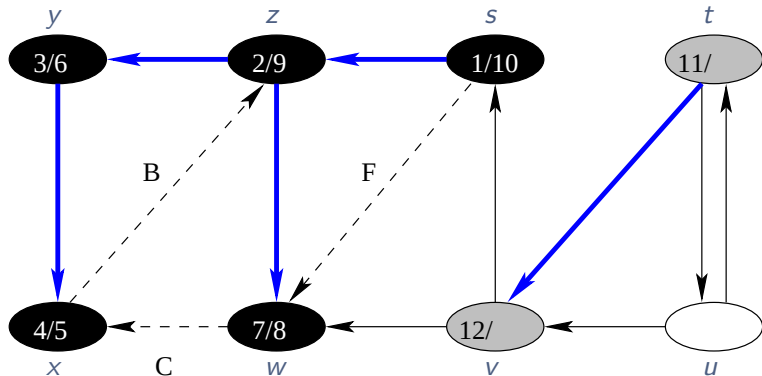
Exemplo de busca em profundidade



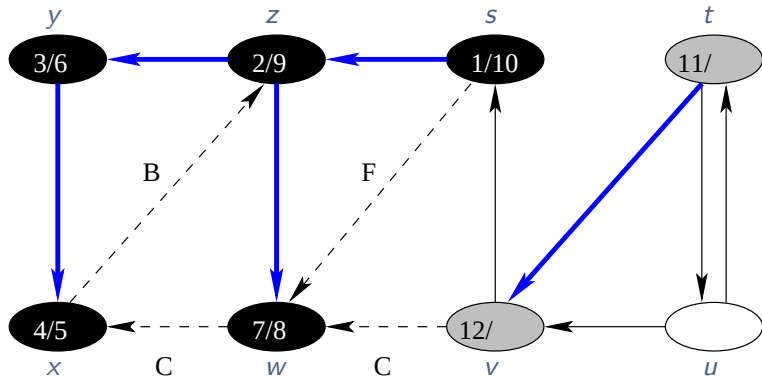
Exemplo de busca em profundidade



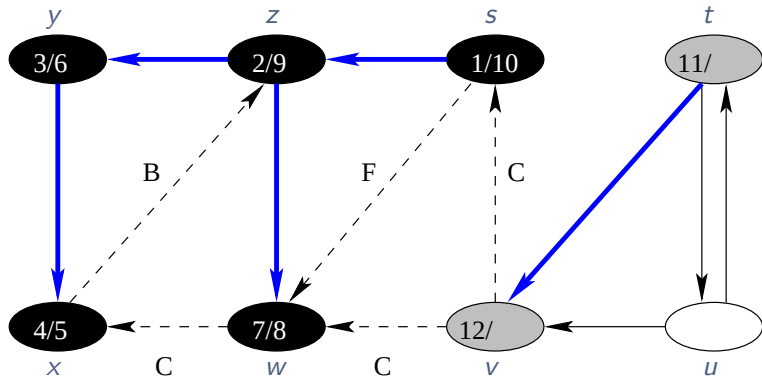
Exemplo de busca em profundidade



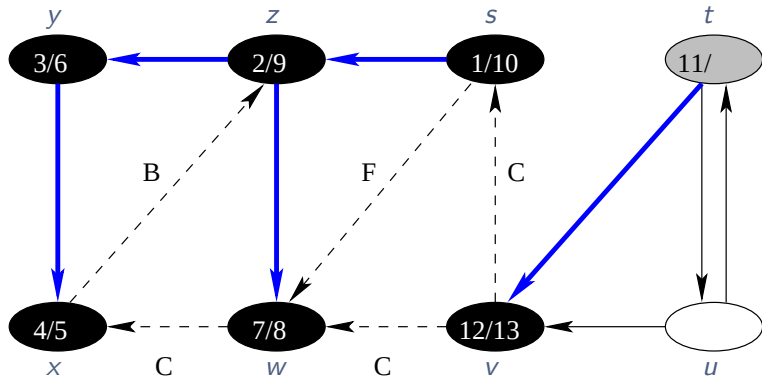
Exemplo de busca em profundidade



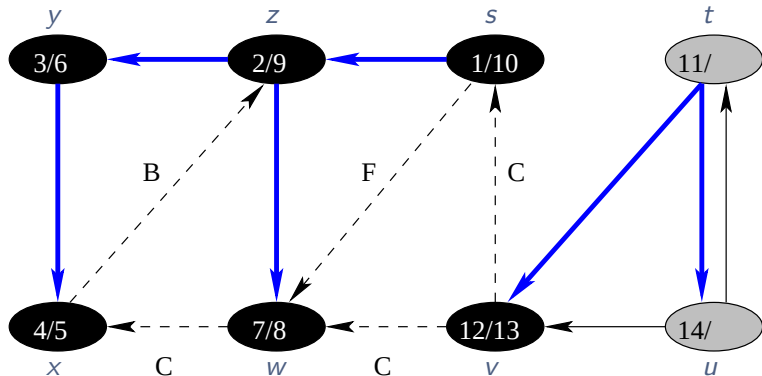
Exemplo de busca em profundidade



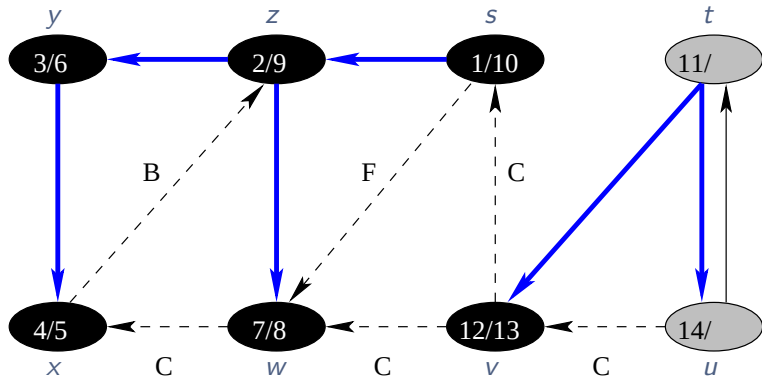
Exemplo de busca em profundidade



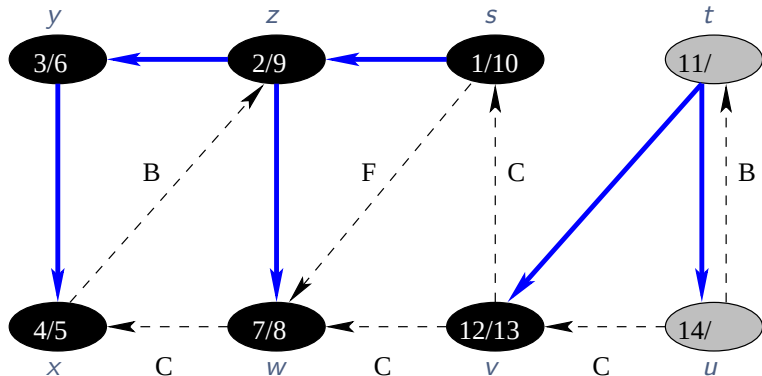
Exemplo de busca em profundidade



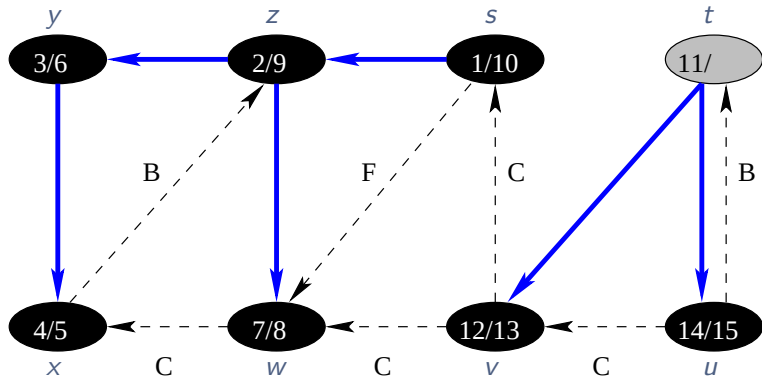
Exemplo de busca em profundidade



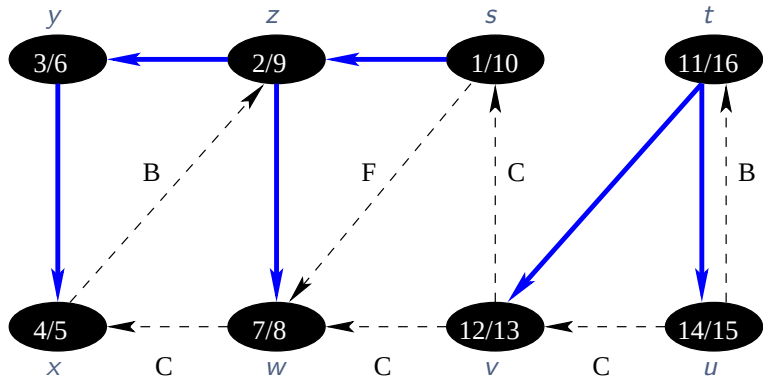
Exemplo de busca em profundidade



Exemplo de busca em profundidade



Exemplo de busca em profundidade



Observe que para todo vértice v

- ▶ v é branco antes do instante $d[v]$
- ▶ v é cinza entre os instantes $d[v]$ e $f[v]$
- ▶ v é preto após o instante $f[v]$

Algoritmo DFS

```
DFS( $G$ )  
1  para cada  $u \in V(G)$  faça  
2       $cor[u] = \text{branco}$   
3       $\pi[u] = \text{NIL}$   
4  tempo = 0  
5  para cada  $u \in V(G)$  faça  
6      se  $cor[u] == \text{branco}$  então  
7          DFS-VISIT( $u$ )
```

- ▶ representamos G com listas de adjacências
- ▶ a floresta de busca em profundidade é representada por π
- ▶ calcula os instantes $d[v]$ e $f[v]$

Algoritmo DFS-VISIT

DFS-visit(u)

```
1  tempo = tempo + 1
2   $d[u]$  = tempo
3   $cor[u]$  = cinza
4  para cada  $v \in Adj[u]$  faça
5      se  $cor[v] ==$  branco então
6           $\pi[v] = u$ 
7          DFS-VISIT( $v$ )
8  tempo = tempo + 1
9   $f[u]$  = tempo
10  $cor[u]$  = preto
```

- constrói uma árvore de busca com origem u

Análise de complexidade

Analizamos o tempo do algoritmo principal DFS

- ▶ a inicialização consome tempo $O(V)$
- ▶ realizamos $|V|$ chamadas a DFS-VISIT

E o tempo da sub-rotina DFS-VISIT

- ▶ processamos cada vértice exatamente uma vez
- ▶ cada chamada percorre sua lista de adjacências
- ▶ o tempo gasto percorrendo adjacências é $O(E)$

A complexidade da busca em profundidade é $O(V + E)$

Propriedades da Busca em Profundidade

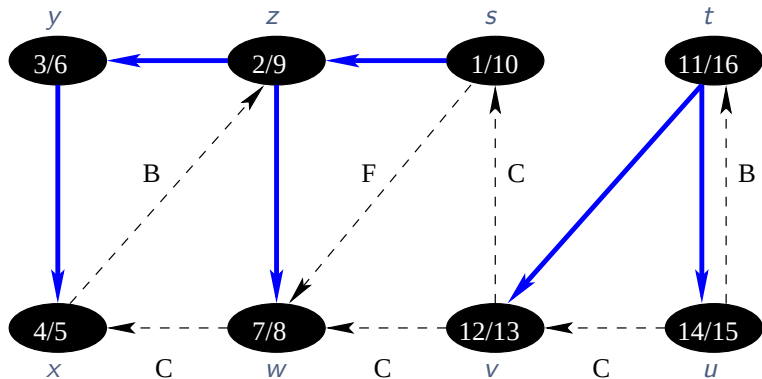
Estrutura de parênteses

Teorema dos parênteses

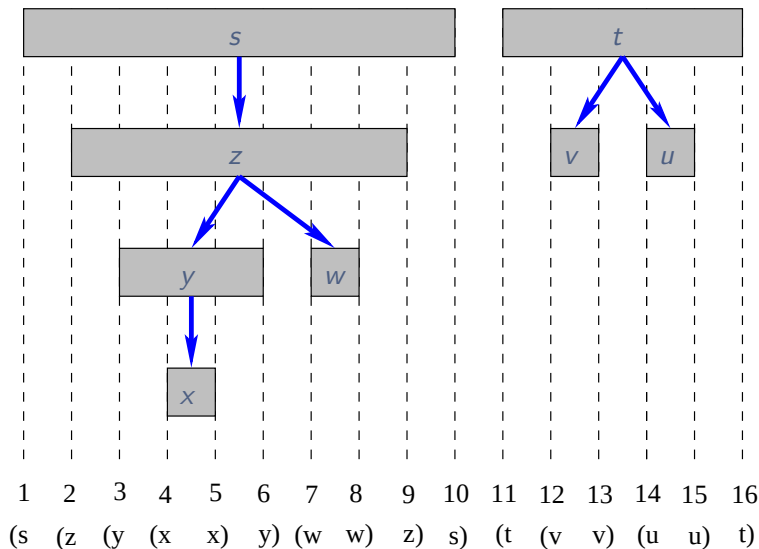
Se u e v são vértices de uma árvore de busca em profundidade, então ocorre exatamente um entre os três casos abaixo:

1. (a) os intervalos $[d[u], f[u]]$ e $[d[v], f[v]]$ são disjuntos
(b) nesse caso u e v não são descendentes um do outro
2. (a) o intervalo $[d[u], f[u]]$ está contido em $[d[v], f[v]]$
(b) nesse caso u é descendente de v
3. (a) o intervalo $[d[v], f[v]]$ está contido em $[d[u], f[u]]$
(b) nesse caso v é descendente de u

Exemplo de floresta de busca



Exemplo de estrutura de parênteses



Demonstração do teorema dos parênteses

- ▶ Sem perda de generalidade, podemos supor que $d[u] < d[v]$.
- ▶ analisamos dois casos:

Caso 1: suponha que $d[v] < f[u]$

- ▶ então v foi descoberto enquanto u era cinza, o que implica que v é um descendente de u
- ▶ e a chamada recursiva para v termina antes da de u , ou seja, $f[v] < f[u]$
- ▶ neste caso, $[d[v], f[v]]$ está contido em $[d[u], f[u]]$

Caso 2: suponha que $f[u] < d[v]$

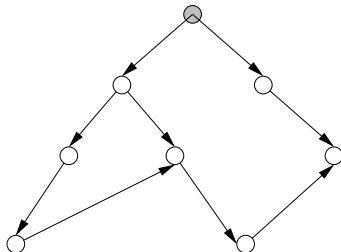
- ▶ então u foi finalizado enquanto v era branco
- ▶ e a chamada de u termina antes que a de v comece
- ▶ portanto u e v não são descendentes um do outro
- ▶ neste caso, $[d[v], f[v]]$ e $[d[u], f[u]]$ são disjuntos. ■

Vértices alcançáveis

Teorema do caminho branco

Considere dois vértices u e v . As seguintes afirmações são equivalentes:

- (1) v é descendente de u na floresta de busca
- (2) quando u foi descoberto, existia um caminho de u a v formado apenas por vértices brancos



Prova do teorema do caminho branco

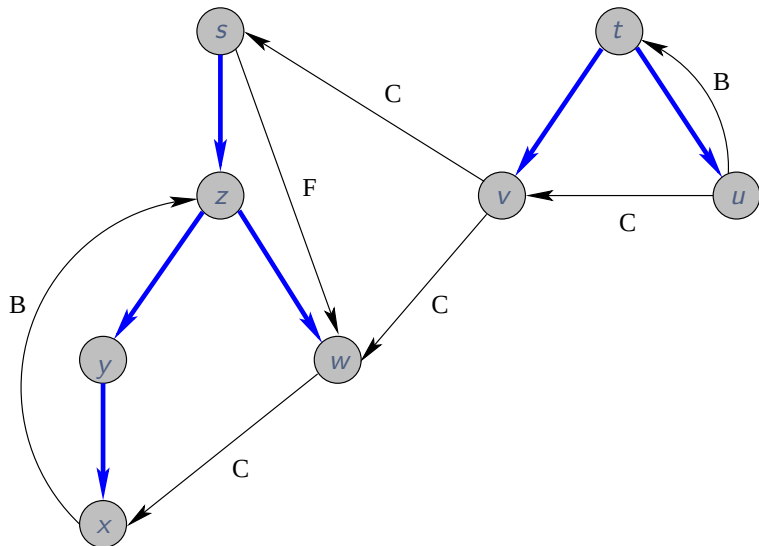
- ▶ (1) $\Rightarrow$ (2)
 - ▶ suponha que v é um descendente próprio de u
 - ▶ pelo Teorema dos Parênteses (T.P.), $d[u] < d[v]$
 - ▶ portanto, v é branco no tempo $d[u]$
 - ▶ Como v pode ser qualquer descendente de u , todos os vértices no caminho de u a v na árvore de busca em profundidade eram brancos no tempo $d[u]$
- ▶ (2) $\Rightarrow$ (1)
 - ▶ considere um caminho branco de u a v no instante $d[u]$
 - ▶ suponha que todo vértice no caminho virou descendente de u , com exceção do vértice v
 - ▶ seja w o vértice antecessor de v nesse caminho (w pode ser o próprio u)
 - ▶ como w é descendente de u , temos $f[w] \leq f[u]$ (T.P.)
 - ▶ como v é descoberto depois de u ser descoberto e antes de w ser finalizado, temos $d[u] < d[v] < f[w] \leq f[u]$
 - ▶ Pelo T.P. temos que $[d[v], f[v]]$ está inteiramente contido no intervalo $[d[u], f[u]]$, e v é descendente de u , contradição.

Classificação de arestas

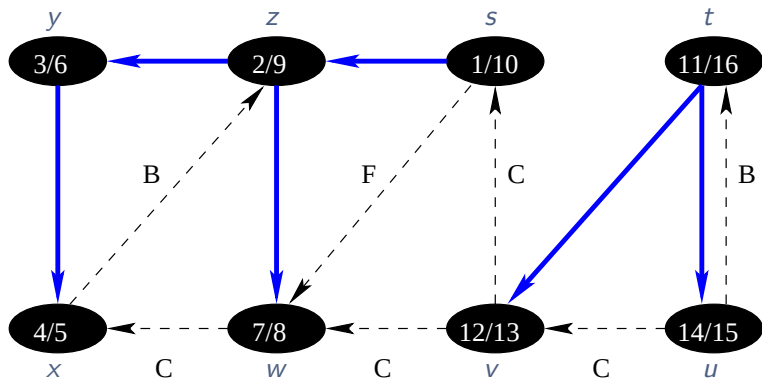
Dada a floresta de busca, podemos classificar arestas do grafo

- ▶ **arestas de árvore** (*tree edges*) são arestas da floresta de busca em profundidade
- ▶ **arestas de retorno** (*back edges*) ligam um vértice a um ancestral. Laços em grafos direcionados são arestas de retorno.
- ▶ **arestas de avanço** (*forward edges*) ligam um vértice a um descendente (não são arestas de árvore)
- ▶ **arestas de cruzamento** (*cross edges*) são todas as outras arestas do grafo

Classificação de arestas



Classificação de arestas



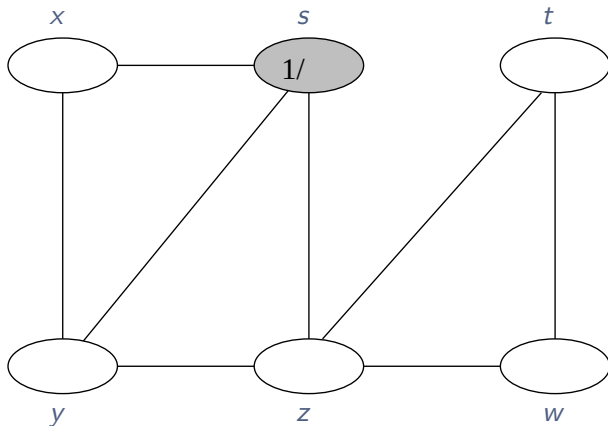
É fácil modificar o algoritmo $\text{DFS}(G)$ para que ele também classifique as arestas de G . (Exercício)

DFS em grafos não direcionados

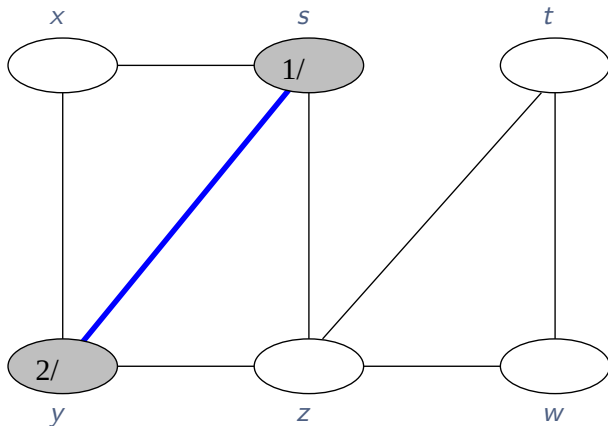
Classificando arestas de um grafo não direcionado:

- ▶ não pode haver aresta de avanço (por quê?)
- ▶ tampouco aresta de cruzamento (por quê?)
- ▶ daí cada aresta é **aresta de árvore** ou **aresta de retorno**

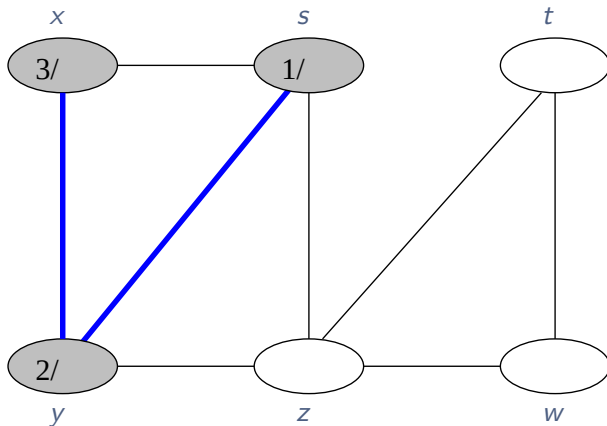
DFS em grafo não direcionado



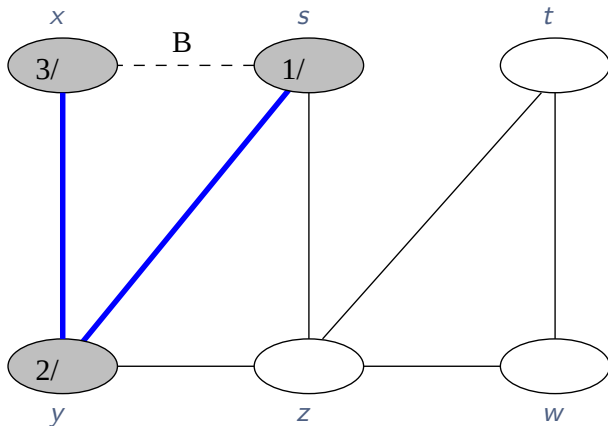
DFS em grafo não direcionado



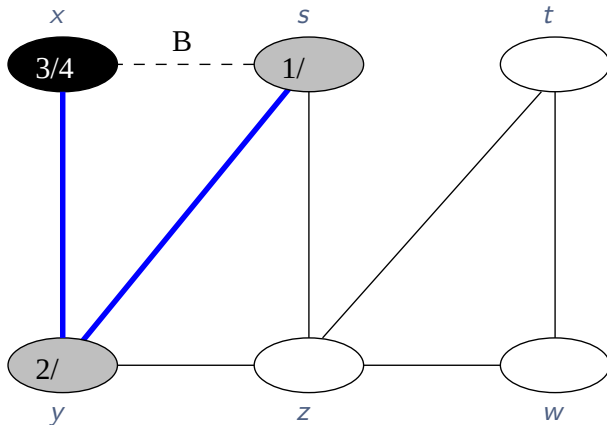
DFS em grafo não direcionado



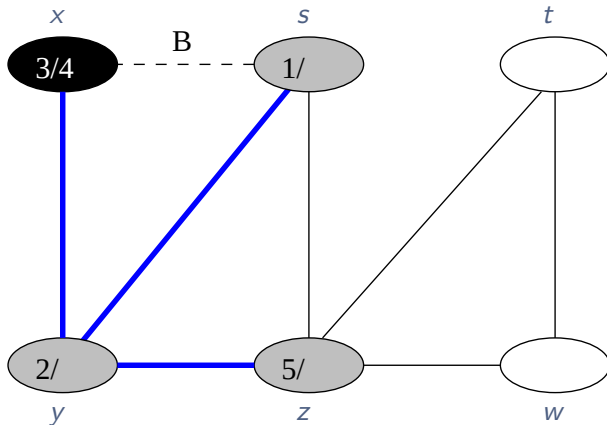
DFS em grafo não direcionado



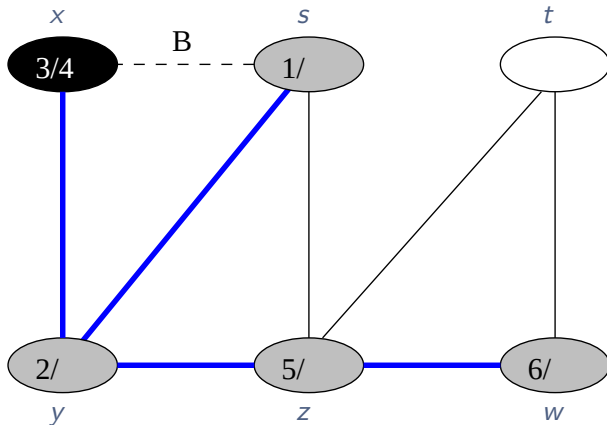
DFS em grafo não direcionado



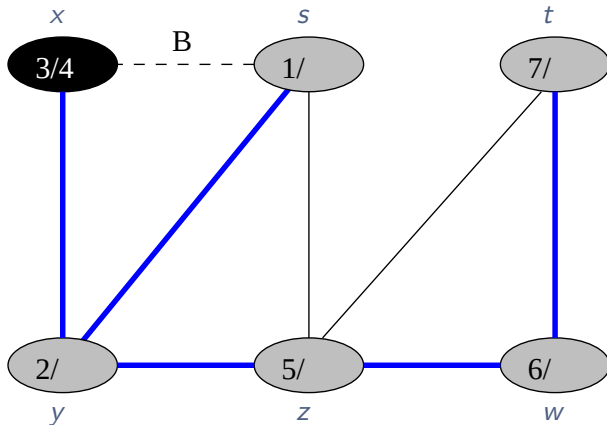
DFS em grafo não direcionado



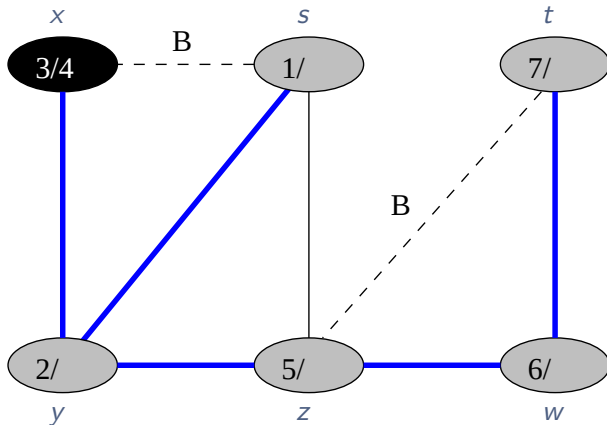
DFS em grafo não direcionado



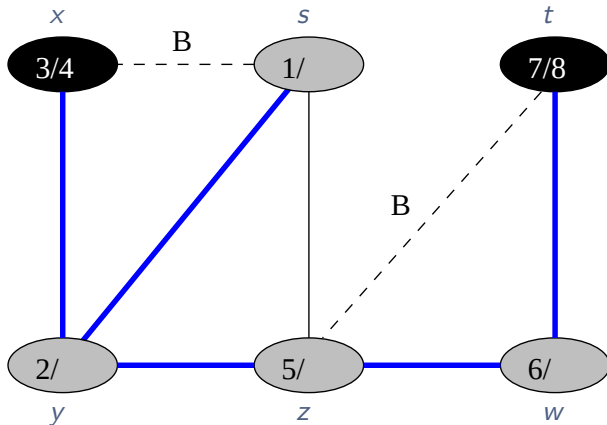
DFS em grafo não direcionado



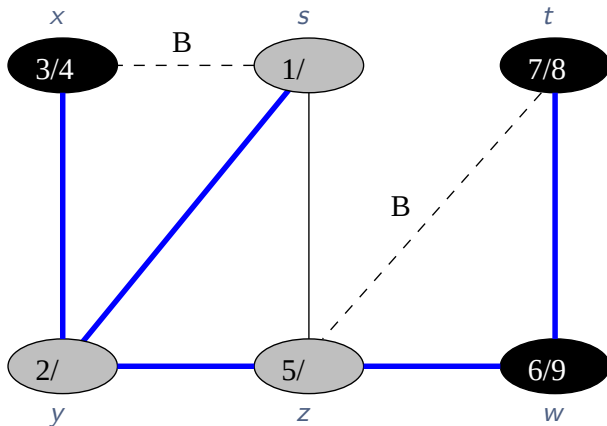
DFS em grafo não direcionado



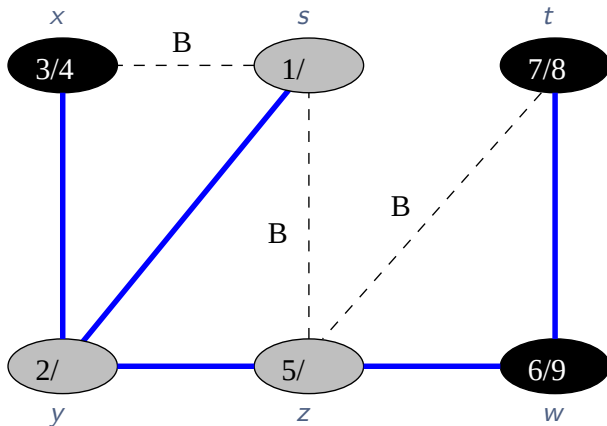
DFS em grafo não direcionado



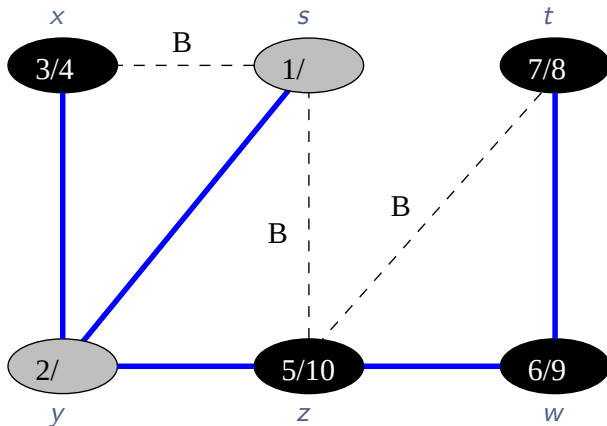
DFS em grafo não direcionado



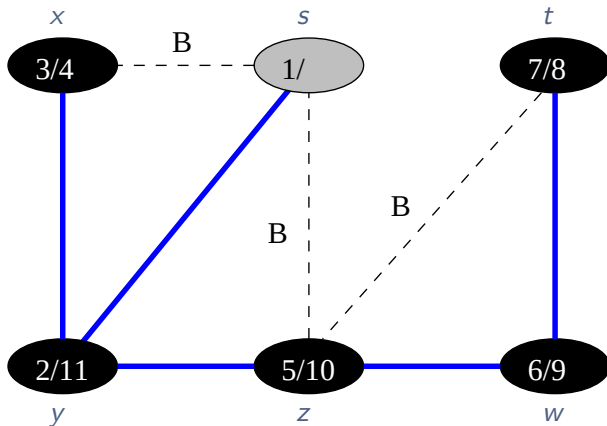
DFS em grafo não direcionado



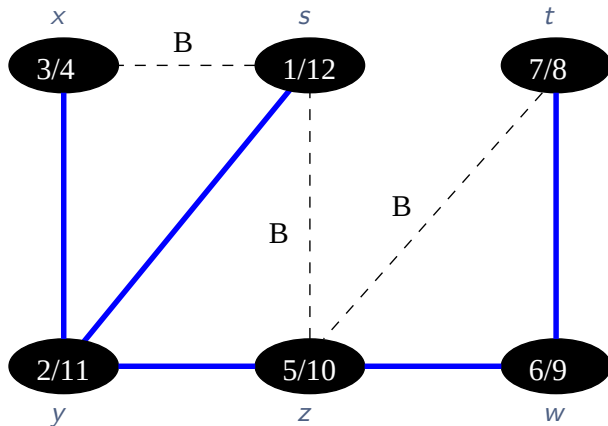
DFS em grafo não direcionado



DFS em grafo não direcionado

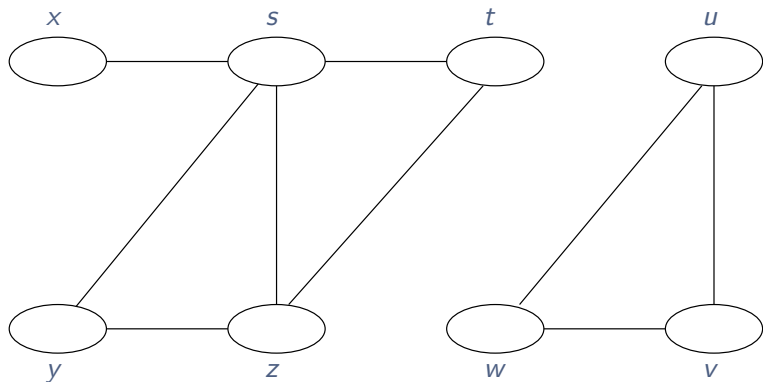


DFS em grafo não direcionado



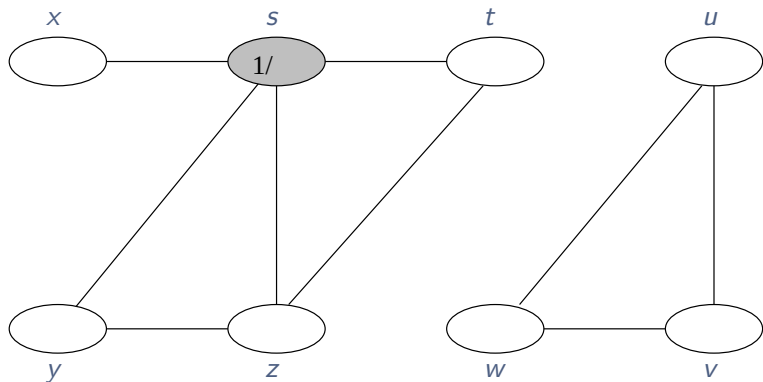
Componentes conexas

Componentes conexas

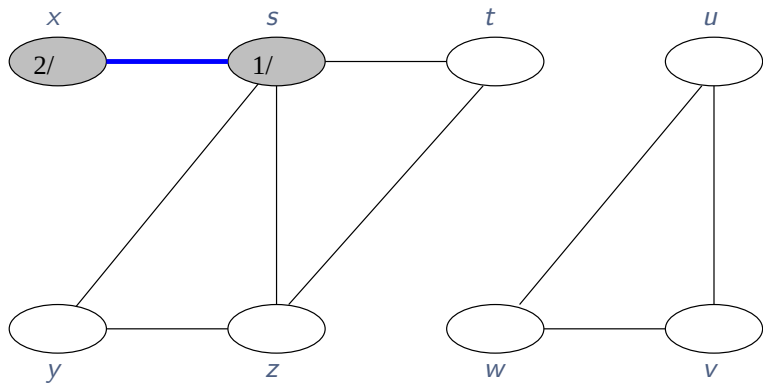


Problema: determinar as componentes conexas de um grafo não direcionado.

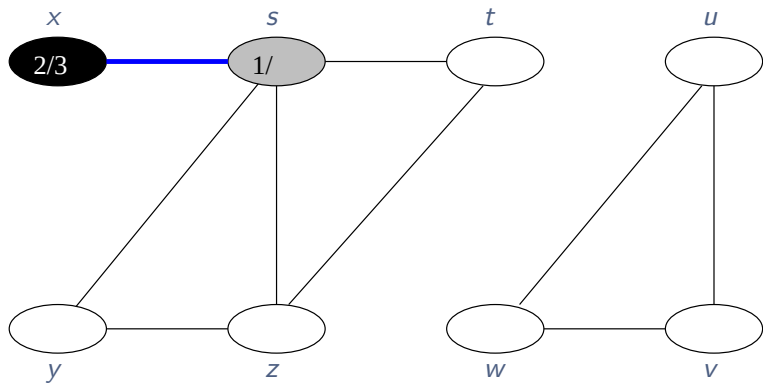
Executando DFS



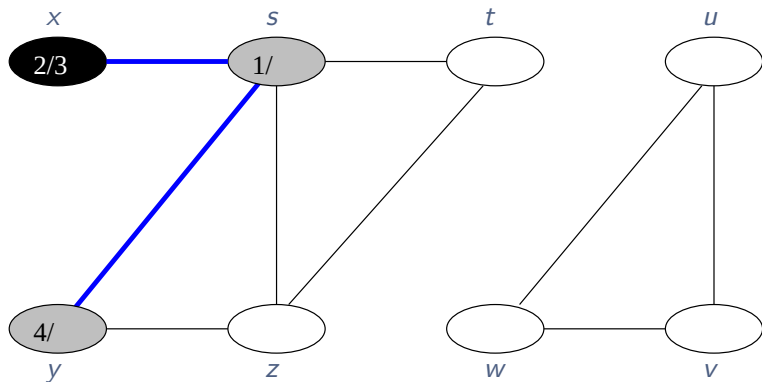
Executando DFS



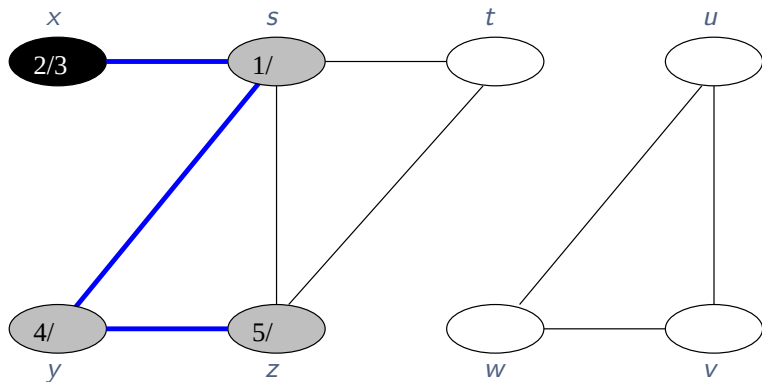
Executando DFS



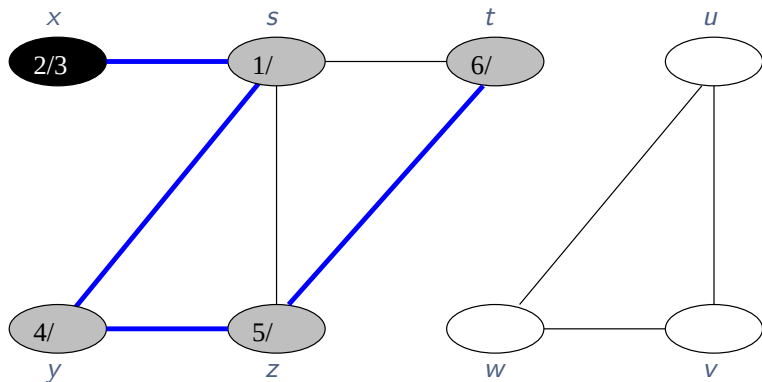
Executando DFS



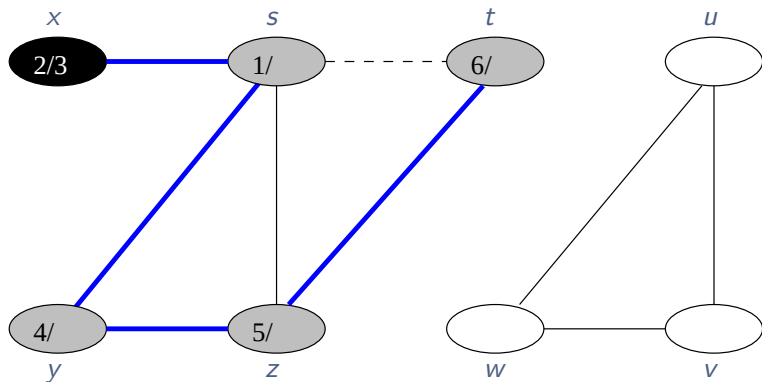
Executando DFS



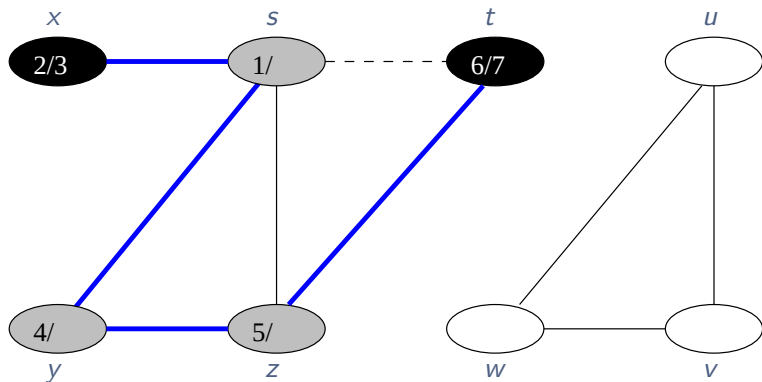
Executando DFS



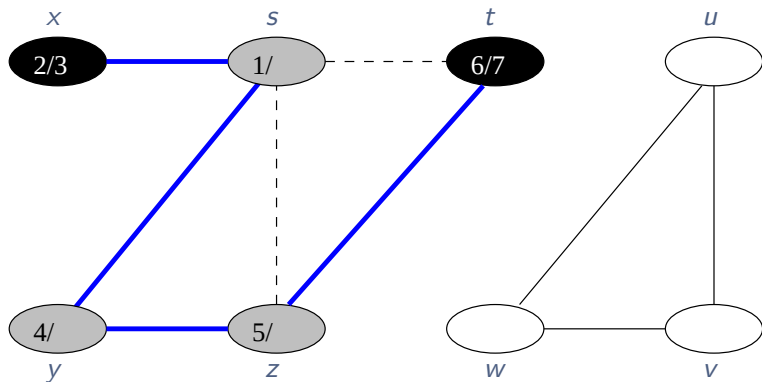
Executando DFS



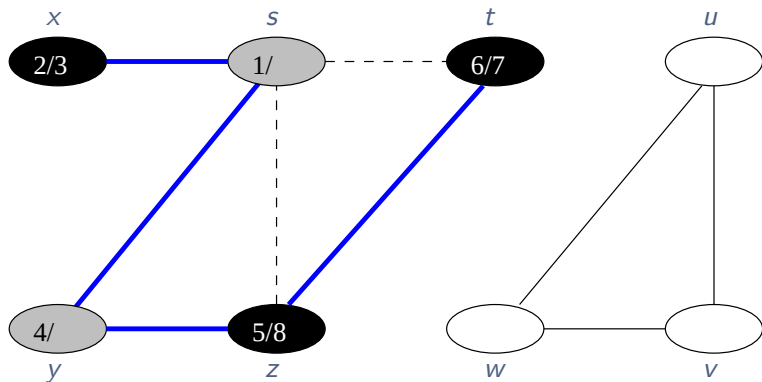
Executando DFS



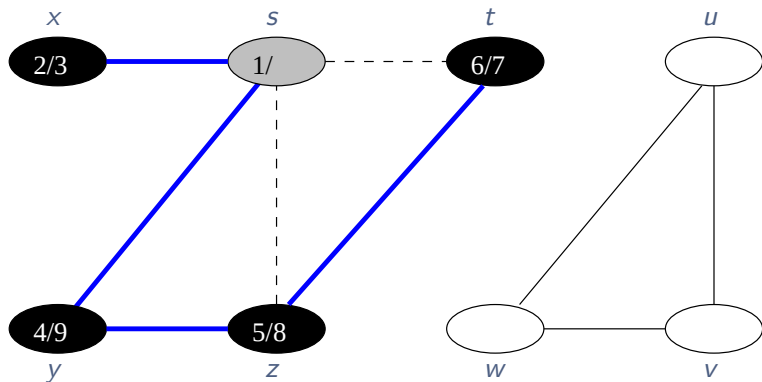
Executando DFS



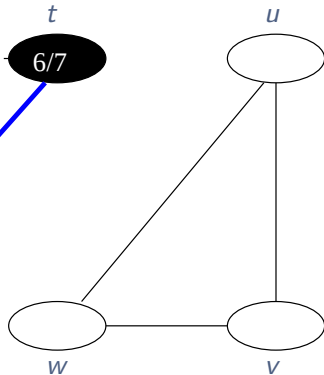
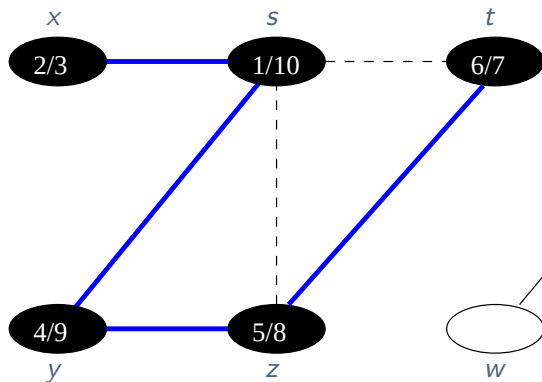
Executando DFS



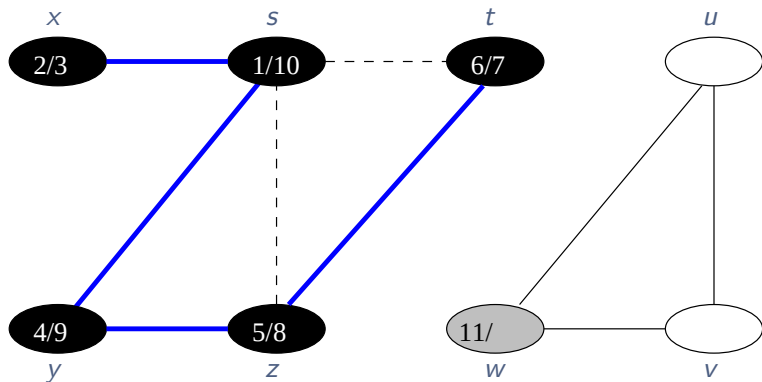
Executando DFS



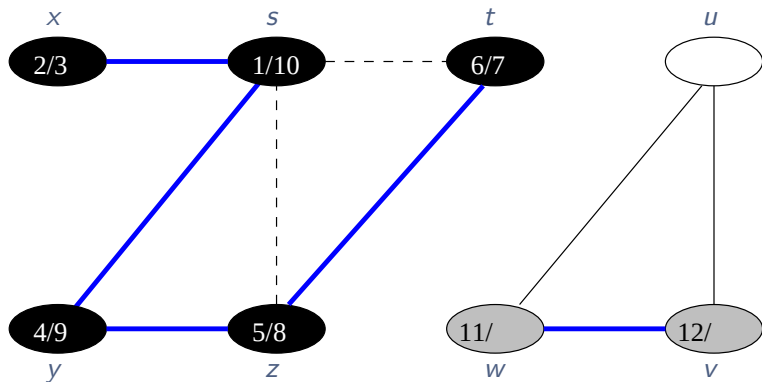
Executando DFS



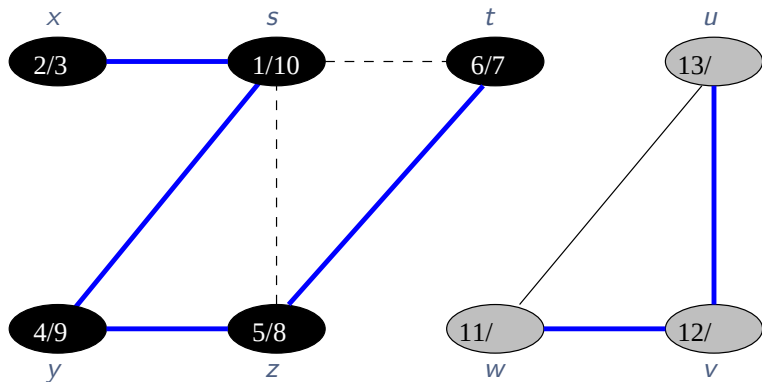
Executando DFS



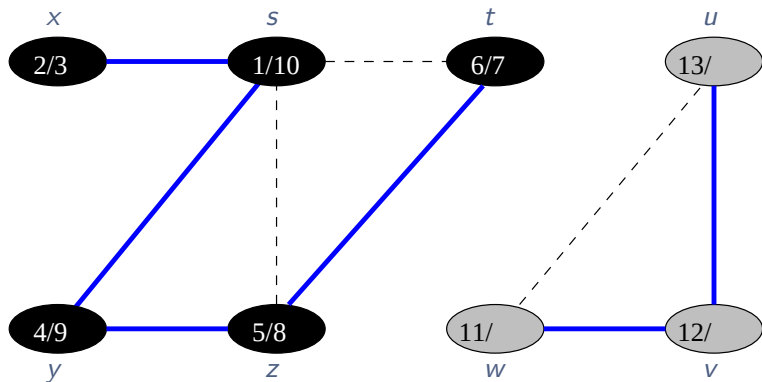
Executando DFS



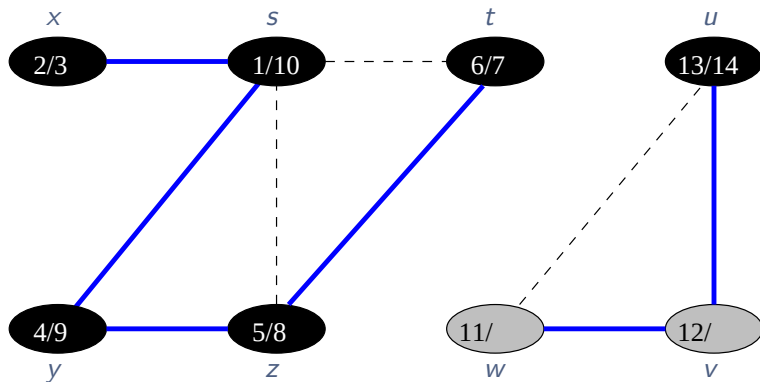
Executando DFS



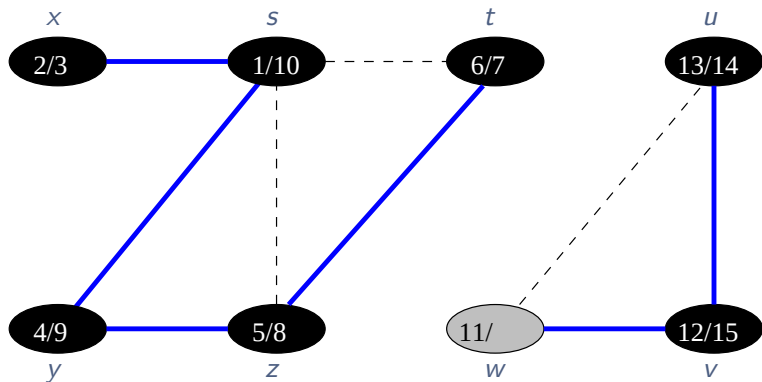
Executando DFS



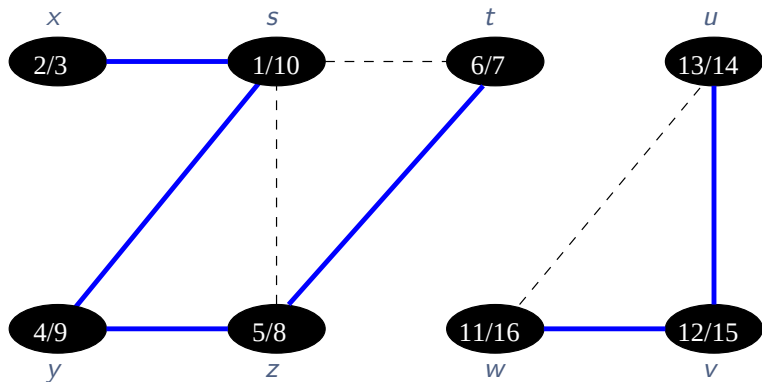
Executando DFS



Executando DFS



Executando DFS



Componentes conexas

Contando o número de componentes

- ▶ cada componente corresponde a uma árvore de busca
- ▶ é o **número de chamadas** a DFS-VISIT a partir de DFS

Vamos modificar DFS

- ▶ identificamos cada componente por um número
- ▶ denotaremos por $comp[v]$ a componente de v

Algoritmo DFS modificado

DFS(G)

```
1  para cada  $u \in V[G]$  faça
2       $cor[u] = \text{branco}$ 
3   $\ell = 0$ 
4  para cada  $u \in V[G]$  faça
5      se  $cor[u] = \text{branco}$  então
6           $\ell = \ell + 1$ 
7          DFS-VISIT( $u$ )
```

- ℓ é o número de chamadas a DFS-VISIT a partir de DFS e é uma variável global

Algoritmo DFS-VISIT modificado

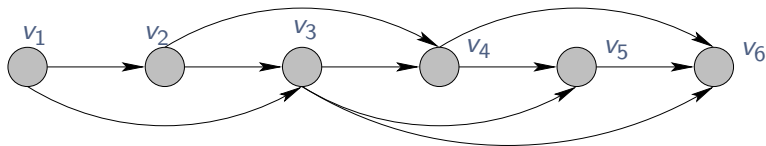
DFS-visit(u)

- 1 $cor[u] = \text{cinza}$
- 2 **para cada** $v \in Adj[u]$ **faça**
- 3 **se** $cor[v] = \text{branco}$ **então**
- 4 DFS-VISIT(v)
- 5 $cor[u] = \text{preto}$
- 6 $comp[u] = \ell$

Ordenação Topológica

Ordenação Topológica

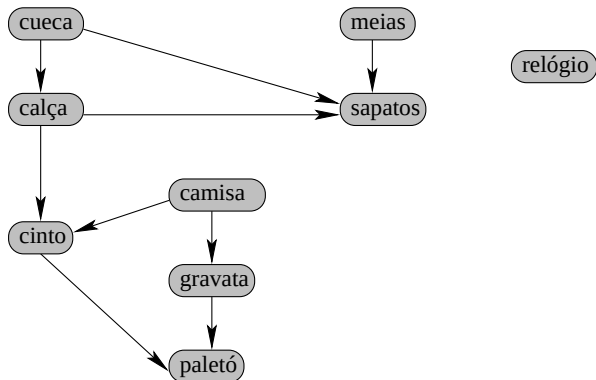
Uma **ordenação topológica** de um grafo direcionado é uma sequência linear $v_1, v_2, \dots, v_n$ dos vértices de G tal que se (v_i, v_j) é uma aresta de G , então v_i aparece antes de v_j na sequência.



Exemplo de aplicação

Representando dependências

- ▶ um grafo pode representar precedências entre tarefas
- ▶ queremos um ordem que respeita as precedências



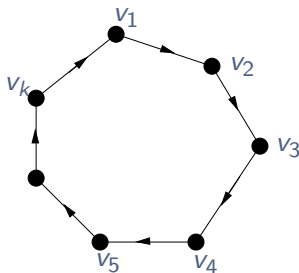
Exemplo de ordenação topológica



Condições de existência

Todo grafo direcionado possui ordenação topológica?

- ▶ **não**, um ciclo direcionado não possui
- ▶ nem outro grafo que contém um ciclo



Seja G um grafo direcionado.

G é **acíclico** se não contiver um ciclo direcionado.

Teorema

Seja G um grafo direcionado.

G é **acíclico** se e somente se possui uma **ordenação topológica**.

Demonstração

- ▶ se G tem uma ordenação topológica, então ele é **acíclico**
- ▶ em seguida, vamos mostrar a recíproca

Um lema auxiliar

- ▶ Uma **fonte** é um vértice com grau de entrada zero.
- ▶ Um **sorvedouro** é um vértice com grau de saída zero.

Lema

Todo grafo direcionado acíclico G com pelo menos um vértice possui uma **fonte** e um **sorvedouro**.

Demonstração

- ▶ tome um caminho mais longo no grafo P que vai de s até t
- ▶ observe que s é uma fonte e t é um sorvedouro

Prova do teorema

Agora podemos terminar a demonstração. Queremos provar que se G é direcionado e acíclico, então G possui ordenação topológica.

- ▶ considere um grafo direcionado acíclico $G = (V, E)$
- ▶ afirmamos que G possui uma ordenação topológica
- ▶ vamos mostrar por indução em $|V|$
- ▶ se $|V| = 1$, então a afirmação é clara

Considere um grafo com pelo menos dois vértices

- ▶ pelo lema anterior, G possui uma fonte v_1
- ▶ pela hipótese de indução, o grafo $G - v_1$ possui uma ordenação topológica $v_2, \dots, v_n$
- ▶ logo $v_1, v_2, \dots, v_n$ é uma ordenação topológica de G

Encontrando uma ordenação topológica

A demonstração anterior é construtiva

- ▶ é baseada em exibir uma ordenação topológica
- ▶ ela sugere um **algoritmo recursivo**

Algoritmo para ordenação topológica

1. encontre uma fonte v_1 de G
 2. recursivamente, obtenha ordenação $v_2, \dots, v_n$ de $G - v_1$
 3. devolva $v_1, v_2, \dots, v_n$
- ▶ pode-se implementar esse algoritmo em tempo $O(V + E)$ (exercício)

Algoritmo baseado em DFS

Considere um grafo direcionado acíclico

- ▶ como não há ciclo, não existe aresta de retorno
- ▶ considere o instante em que v fica preto
- ▶ nesse instante **todos** seus vizinhos são pretos
- ▶ isso sugere considerar os vértices na ordem de término

Ideia para o algoritmo

- ▶ o primeiro vértice a ficar preto não tem arestas saindo
- ▶ o segundo só pode ter arestas para o primeiro
- ▶ o terceiro só pode ter arestas para os dois primeiros
- ▶ etc.

Algoritmo TOPOLOGICAL-SORT

Topological-Sort(G)

- 1 execute DFS(G) e calcule $f[v]$ para cada vértice v
- 2 quando um vértice finalizar, insira-o no **início** de uma lista
- 3 devolva a lista resultante

- ▶ inserir cada um dos $|V|$ vértices leva tempo $O(1)$
- ▶ além disso, executamos DFS uma vez
- ▶ portanto, a complexidade de tempo é $O(V + E)$

Exemplo



Teorema

TOPOLOGICAL-SORT(G) devolve ordenação topológica de um grafo direcionado acíclico G .

Demonstração

- ▶ a lista devolvida está em ordem **decrecente** de $f[v]$
- ▶ considere uma aresta arbitrária (u, v)
- ▶ basta mostrar que $f[u] > f[v]$
- ▶ considere o instante em que (u, v) foi examinada
- ▶ como (u, v) não é aresta de retorno, v não pode ser cinza
 1. se v for branco, ele será descendente de u e $f[u] > f[v]$
 2. se v for preto, então ele já foi finalizado e $f[u] > f[v]$
- ▶ em qualquer caso, obtemos o que desejávamos. ■